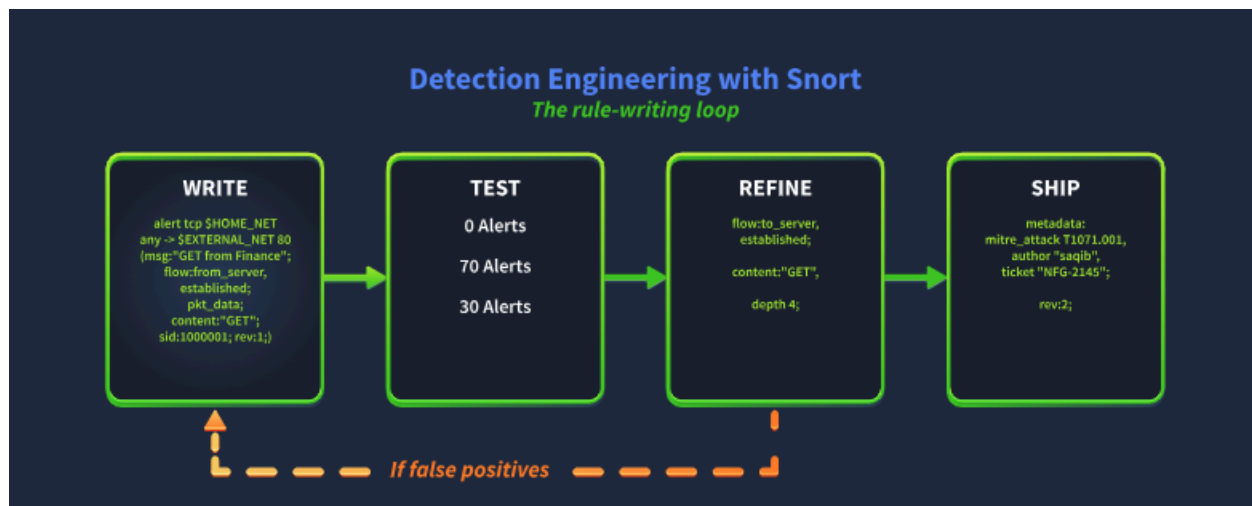


L2 - 12.2. Detection Engineering With Snort

Task 1 - Introduction

This room is about writing that rule, along with the rest of the rules we should have had in place. The rule syntax carries over. The config file is now `snort.lua` instead of `snort.conf`, and preprocessors are called inspectors.

Task 2 - Advanced Rule Options



- `task-02-rule-anatomy.pcap`: A mixed-traffic capture from the Finance subnet. Outbound HTTP browsing, outbound POST requests to form handlers, HTTPS traffic, and DNS queries.
- `task-02-starter.rules`: The rule we are about to test. It is supposed to alert on outbound HTTP GET requests from Finance workstations, but something about it is off.

Let's move into the lab directory once, so every command from here on stays short:

```
ubuntu@tryhackme:~$ cd /home/ubuntu/lab/task-02
```

```
ubuntu@tryhackme:~/lab/task-02$
```

Now let's open the rule and see what we have been given:

```
ubuntu@tryhackme:~/lab/task-02$ cat task-02-starter.rules
alert tcp $HOME_NET any -> $EXTERNAL_NET 80 ( \
  msg:"L2-SNORT Outbound HTTP GET from Finance subnet"; \
  flow:from_server,established; \
  pkt_data; content:"GET"; \
  sid:1000101; \
  rev:1; \
)
ubuntu@tryhackme:~/lab/task-02$
```

First Run: No Alerts

```
ubuntu@tryhackme:~/lab/task-02$ sudo snort -c /etc/snort/snort.lua -R task-02-starter.rules -r task-02-rule-anatomy.pcap -A alert_fast -q
ubuntu@tryhackme:~/lab/task-02$
```

The output comes back empty, even though the PCAP contains plenty of outbound HTTP GET traffic from Finance workstations. Something in the rule is preventing Snort from running the content match at all.

Fixing the Flow Direction: Let's fix it. Open the rule file in our editor and change `flow:from_server,established;` to `flow:to_server,established;` and save the file.

```
ubuntu@tryhackme:~/lab/task-02$ nano task-02-starter.rules
alert tcp $HOME_NET any -> $EXTERNAL_NET 80 ( \
  msg:"L2-SNORT Outbound HTTP GET from Finance subnet"; \
  flow:to_server,established; \
  pkt_data; content:"GET"; \
  sid:1000101; \
  rev:1; \
)

```

Re-run Snort, but instead of just counting alerts we will group them by source address, so we can see which workstations the rule is firing on:

```
ubuntu@tryhackme:~/lab/task-02$ sudo snort -c /etc/snort/snort.lua -R task-02-starter.rules -r task-02-rule-anatomy.pcap -A alert_fast -q | grep -oE '10\.14\.22\.[0-9]+' | sort | uniq -c
      5 10.14.22.156
     80 <REDACTED>
      5 10.14.22.45
     20 10.14.22.88
```

Looking at the output, four source addresses are firing this rule. Three of them, `10.14.22.88`, `10.14.22.45`, and `10.14.22.156`, are the Finance workstations we expect to make outbound GET requests, and together they represent all the legitimate outbound GET activity in the capture.

Tightening the Content Match: The rule fires on both POST and GET requests. `content:"GET";` tells Snort to fire on any payload containing the three bytes G, E, T. Some POST bodies in the PCAP carry the literal string `GET` inside form data. `depth N` restricts the match to the first N bytes, and `offset N` shifts the search window to start at byte N. In Snort 3, modifiers attach to the content option with commas, and value-bearing modifiers use a space rather than a colon, so we write `depth 4`, not `depth:4`. A real HTTP GET request starts with the bytes `GET` (with a trailing space) at offset 0. Open the rule again and change `content:"GET";` to `content:"GET ", depth 4;`, as shown below:

```
ubuntu@tryhackme:~/lab/task-02$ nano task-02-starter.rules
alert tcp $HOME_NET any -> $EXTERNAL_NET 80 ( \
  msg:"L2-SNORT Outbound HTTP GET from Finance subnet"; \
  flow:to_server,established; \
  pkt_data; content:"GET ", depth 4; \
  sid:1000101; \
  rev:1; \
)
```

Save the file and re-run Snort against the same PCAP with the same source grouping applied:

```
ubuntu@tryhackme:~/lab/task-02$ sudo snort -c /etc/snort/snort.lua -R task-02-starter.rules -r task-02-rule-anatomy.pcap -A alert_fast -q | grep -oE '10\.14\.22\.[0-9]+' | sort | uniq -c
```

```
5 10.14.22.156
5 10.14.22.45
20 10.14.22.88
```

Looking at the output, the POST source `<REDACTED>` has been dropped from the table. Only the three Finance workstations that genuinely make outbound GET requests remain, and the counts match the genuine outbound GET traffic in the capture. The false positives are gone because POST payloads begin with the bytes `POST`, not `GET`, and depth 4 prevents the content engine from looking past the first 4 bytes.

Wrapping It up With Metadata: Common fields we populate include:

- `mitre_attack` for the ATT&CK technique
- `author` for rule ownership
- `ticket` for the internal reference explaining its existence
- `created` or `updated` for change tracking.

Validate Answer Rule

```
ubuntu@tryhackme:~/lab/task-02$ sudo snort -c /etc/snort/snort.lua -R task-02-answer.rules -r task-02-rule-anatomy.pcap -A alert_fast -q | wc -l
```

```
<REDACTED>
```

Looking at the output, all three Finance workstations are represented, the POST source is gone, and the metadata is embedded in the rule body. The flow direction is correct, the content clause is anchored to the first 4 bytes, and the attribution is in place; this is the rule we ship.

Answer the questions below

Which workstation fires this rule most often after fixing flow direction, despite never making an outbound GET request?

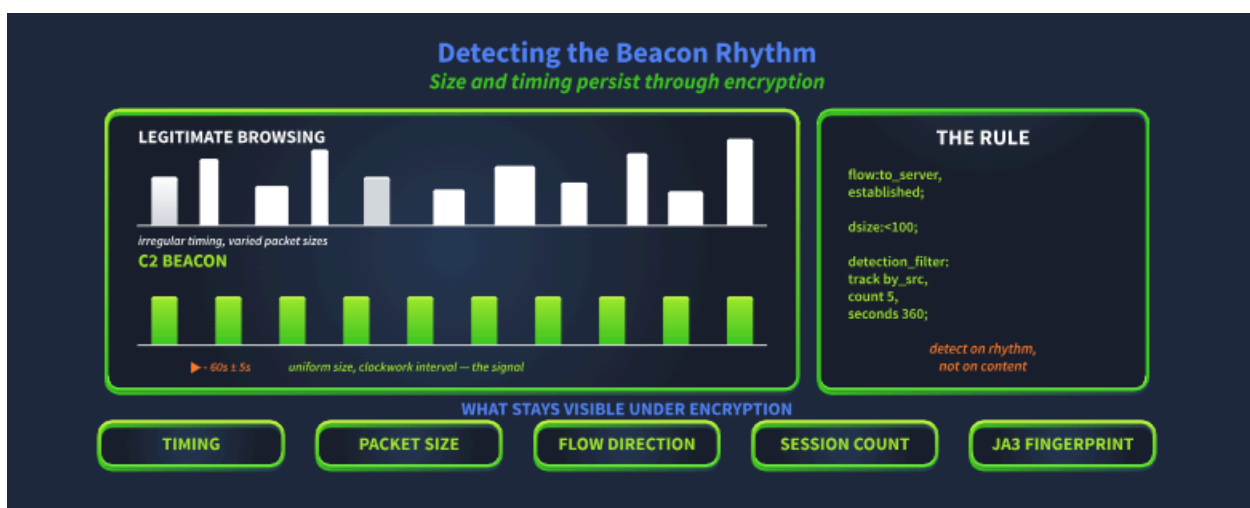
10.14.22.202

```
ubuntu@tryhackme:~/lab/task-02$ sudo snort -c /etc/snort/snort.lua -R task-02-starter.rules -r task-02-rule-anatomy.pcap -A alert_fast -q | grep -oE '10\.14\.22\.[0-9]+' | sort | uniq -c
   5 10.14.22.156
  80 10.14.22.202
   5 10.14.22.45
  20 10.14.22.88
ubuntu@tryhackme:~/lab/task-02$ nano task-02-starter.rules
ubuntu@tryhackme:~/lab/task-02$ sudo snort -c /etc/snort/snort.lua -R task-02-starter.rules -r task-02-rule-anatomy.pcap -A alert_fast -q | grep -oE '10\.14\.22\.[0-9]+' | sort | uniq -c
   5 10.14.22.156
   5 10.14.22.45
  20 10.14.22.88
```

After tightening the content match to `content:"GET ", depth 4`, how many alerts does the rule produce?

30

Task 3 - Behaviour-Based Detection



- **Lab Orientation:** Under `/home/ubuntu/lab/task-03/`, we have three files waiting for us:
- `task-03-beaconing.pcap` : A 30-minute capture of Finance subnet egress. WKST-FINANCE-04 beacons to `194.165.16.56` on TCP/443 every 60 seconds with 5-second jitter, mixed with legitimate HTTPS browsing from neighbouring workstations.
- `task-03-starter.rules` : A first-pass rule that alerts on the C2 IP directly. It works on this capture, but it is the kind of rule that ages badly.
- `task-03-answer.rules` : Empty. We will build the behaviour-based rule here.

From Signatures to Behaviour: Behaviour-based detection targets what the attacker does rather than what they currently hold. A workstation reaching out to the same external host every 60 seconds is not casual browsing or an update check; the pattern is automation, and the attacker can rotate the destination tomorrow but cannot disguise the timing. **Writing the Beacon Rule:** Let's open the starter rule and see what we are replacing:

```
ubuntu@tryhackme:~/lab/task-03$ cat task-03-starter.rules
alert tcp $HOME_NET any -> 194.165.16.56 any ( \
    msg:"L2-SNORT SILENT TRANSFER C2 IP match (signature-only)"; \
    flow:to_server,established; \
    sid:1000001; \
    rev:1; \
)
```

Looking at the output, it is a straight IP match: `alert tcp $HOME_NET any -> 194.165.16.56 any`. It fires when this specific C2 address shows up on our wire. The moment the attacker rotates to a new IP, the rule misses the next beacon.

Validating the Detection: Let's run the rule against the PCAP and count the alert line

```
ubuntu@tryhackme:~/lab/task-03$ sudo snort -c /etc/snort/snort.lua -R task-03-answer.rules -r task-03-beaconing.pcap -A al
```

```
ert_fast -q
4/13-09:21:01.532649 [**] [1:1000002:1] "POSSIBLE C2 BEACON -
Finance egress regularity" [**] [Priority: 0] {TCP} <REDACTED
>:49153 -> 194.165.16.56:443
04/13-09:21:55.842322 [**] [1:1000002:1] "POSSIBLE C2 BEACON
- Finance egress regularity" [**] [Priority: 0] {TCP} <REDACT
ED>:49154 -> 194.165.16.56:443
```

Looking at the output, the detection filter is doing its job: it ignores the first four beacons of each cluster and alerts on every fifth across the 30-minute capture. The legitimate HTTPS browsing sessions in the same capture never produce packets of that size at that rhythm, so they stay quiet.

Let's confirm the source on each alert:

```
ubuntu@tryhackme:~/lab/task-03$ sudo snort -c /etc/snort/snort.
lua -R task-03-answer.rules -r task-03-beaconing.pcap -A al
ert_fast -q | grep -oE '10\.14\.22\.[0-9]+' | sort -u
<REDACTED>
```

The output shows a single source, which is WKST-FINANCE-04. The neighboring workstations browsing the web never cross the threshold, which is exactly the signal we wanted.

Where Signatures Stop Working: Our rule works because a TLS heartbeat leaks size and timing even when the payload is encrypted, but not every modern protocol exposes those features. Several recent developments have reduced or eliminated the cleartext fields that content-matching rules depend on:

- **QUIC (UDP/443):** Both the transport negotiation and the TLS handshake are encrypted before any application data is exchanged, leaving no cleartext SNI, no visible method field, and no URI to inspect. From a `content` rule perspective, the wire looks identical regardless of what the encrypted payload contains, and there is simply nothing to match against.
- **DoH (DNS over HTTPS):** DNS resolution is tunnelled inside a standard HTTPS request over TCP/443. Rules built to inspect UDP/53 traffic or parse DNS header structures will never see the query name, because the query never

travels as DNS; it travels as encrypted HTTP body content. Domain-based blocklist detection at the network layer becomes effectively blind.

- **ECH (Encrypted Client Hello):** In standard TLS, the SNI field in the ClientHello is sent in cleartext, giving detection rules a reliable hook for domain matching. ECH encrypts that entire initial handshake message. Any rule matching `content:"update.softpatch-cdn.com"` in the ClientHello will never fire; the field it expects to read is not accessible in plaintext.
- **IP reputation decay:** Threat intelligence blocklists are a snapshot in time. An IP address flagged as active C2 infrastructure six months ago may since have been reclaimed by a CDN, reassigned to a cloud provider, or re-registered by a different operator entirely. Stale entries generate false positives that gradually erode analyst trust in the alert class, and after enough of them the instinct is to suppress rather than triage.

The content is opaque, but the observable properties are still there. Timing cannot be encrypted: a host beaconing every 60 seconds with 5-second jitter produces exactly the same flow pattern whether the underlying protocol is TLS or QUIC. Packet sizes remain visible on the wire regardless of the payload, and flow direction and session frequency are present in every connection record your network visibility tooling produces. Our rule is built entirely on those features, which is why it fires reliably on TLS today and, with the appropriate protocol keyword, will work against QUIC tomorrow. When we cannot see inside the packet, we analyze how the packets behave, and that discipline is at the center of detection engineering on modern encrypted networks.

Answer the questions below

After writing the behaviour-based rule and running it against `task-03-beaconing.pcap`, how many alerts does it produce?

25

What is the source IP address of the host that crosses the detection threshold in `task-03-beaconing.pcap`?

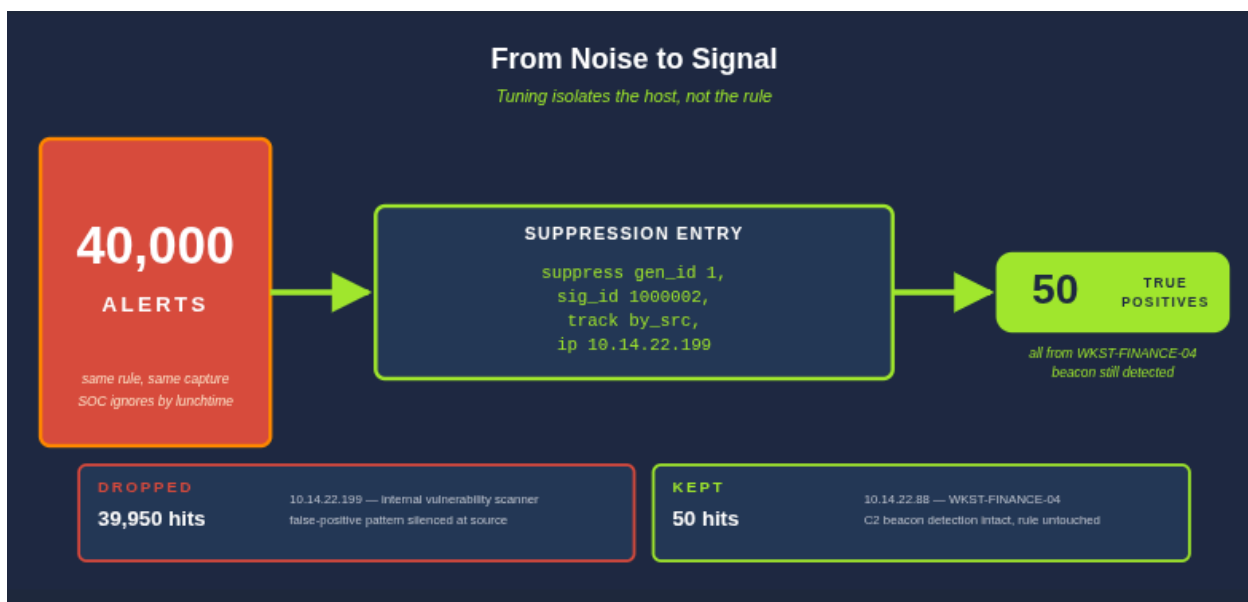
10.14.22.88

```

gularity: [ ] [Priority: 0] {IP} 10.14.22.88.49161 -> 194.163.16.56.443
ubuntu@tryhackme:~/lab/task-03$ sudo snort -c /etc/snort/snort.lua -R task-03-an
swer.rules -r task-03-beaconing.pcap -A alert_fast -q | grep -oE '10\.14\.22\[0
-9\]+' | sort -u
10.14.22.88
ubuntu@tryhackme:~/lab/task-03$ ubuntu@tryhackme:~/lab/task-03$ sudo snort -c /e
tc/snort/snort.lua -R task-03-answer.rules -r task-03-beaconing.pcap -A alert_fa
st -q | wc -l
bash: ubuntu@tryhackme:~/lab/task-03$: No such file or directory
0
ubuntu@tryhackme:~/lab/task-03$ sudo snort -c /etc/snort/snort.lua -R task-03-an
swer.rules -r task-03-beaconing.pcap -A alert_fast -q | wc -l
25
ubuntu@tryhackme:~/lab/task-03$

```

Task 4 - Thresholds, Suppression, Tuning



- `task-04-noisy-traffic.pcap`: an 80 MB capture. It carries the same C2 beacon from WKST-FINANCE-04 at `10.14.22.88`, but it also includes a full day of activity from the internal vulnerability scanner at `10.14.22.199`. The scanner's probes happen to look enough like our beacon to trigger the rule.
- `task-04-ruleset.rules`: a copy of the beacon rule from Task 3 for reference.
- `task-04-answer.rules`: where we will write our tuned rule.
- `task-04-suppress.lua`: an empty Lua suppression file we will populate in the tuning step below.

Counting the Noise: Let's run our rule against the noisy capture and see what we get:

```
ubuntu@tryhackme:~/lab/task-04$ sudo snort -c /etc/snort/snort.lua -R task-04-answer.rules -r task-04-noisy-traffic.pcap -A alert_fast -q | wc -l
<REDACTED>
```

Looking at the output, nearly 40,000 alerts. A SOC receiving close to 40,000 hits from the same rule in a single capture is a SOC that stops reading that rule by lunchtime. Before we touch the rule itself, let's see where the volume is coming from:

```
ubuntu@tryhackme:~/lab/task-04$ sudo snort -c /etc/snort/snort.lua -R task-04-answer.rules -r task-04-noisy-traffic.pcap -A alert_fast -q | grep -oE '10\.14\.22\.[0-9]+' | sort | uniq -c | sort -rn | head
39835 10.14.22.199
52 10.14.22.88
```

The output shows 39,835 hits from `10.14.22.199` and 52 hits from `10.14.22.88`. The scanner is the noise, and WKST-FINANCE-04 is the signal. If we tighten the rule itself by shrinking the size window or stretching the interval, we risk missing the real beacon the next time the attacker changes their jitter. The cleaner move is to silence the scanner without touching the rule's detection logic.

Threshold and Suppression Options: Snort gives us two distinct controls for alert volume. `threshold` caps or gates a rule globally: `threshold type limit` fires at most N times per window per source, `threshold type threshold` requires N hits before firing once, and `threshold type both` combines the two with a cooldown. We reach for thresholds when the rule itself is right, but the traffic is high volume for everyone seeing it. `suppress` works differently. It silences a rule for a specific source, destination, or IP list without changing the rule text. We reach for it when one host is a known false-positive generator and every other source is still a real signal. That is exactly our situation here: `10.14.22.199` is the only source we want to ignore.

Applying the Suppression: The suppression entry goes into `task-04-suppress.lua`, the empty Lua file already in the lab directory. Open it and add the suppress table:

```
ubuntu@tryhackme:~/lab/task-04$ nano task-04-suppress.lua
ubuntu@tryhackme:~/lab/task-04$
```

Add the following:

```
suppress =
{
  { gid = 1, sid = 1000002, track = 'by_src', ip = '10.14.22.199' }
}
```

`gid = 1` targets rules from the text ruleset. `sid = 1000002` is the SID of our beacon rule. `track = 'by_src'` tells Snort to match on the source address. `ip = '10.14.22.199'` is the single host we want to silence. Save and exit.

Re-Running With Suppression: Re-run with the suppression file loaded via `--tweaks`:

```
ubuntu@tryhackme:~/lab/task-04$ sudo snort -c /etc/snort/snort.lua -R task-04-answer.rules -r task-04-noisy-traffic.pcap -A alert_fast -q --tweaks task-04-suppress | wc -l
<REDACTED>
```

Looking at the output, the scanner is silent and only WKST-FINANCE-04 remains, firing on every fifth packet across the capture. We kept the detection logic unchanged and removed a known-good source from the alert stream.

The pattern we just followed is the one we repeat for every noisy rule. Run the rule in alert-only mode in staging, measure volume over 24 to 48 hours, and classify the top sources into legitimate, misconfigured, and true positive.

Apply `suppress` for the legitimate sources and `threshold` for frequency-driven noise. Promote to production and watch for drift as the network changes. The discipline that separates a workable SOC from an ignored one is not rule writing; it is what we do with the rules after they ship.

Answer the questions below

Before adding any suppression entry, how many total alerts does Snort produce against task-04-noisy-traffic.pcap?

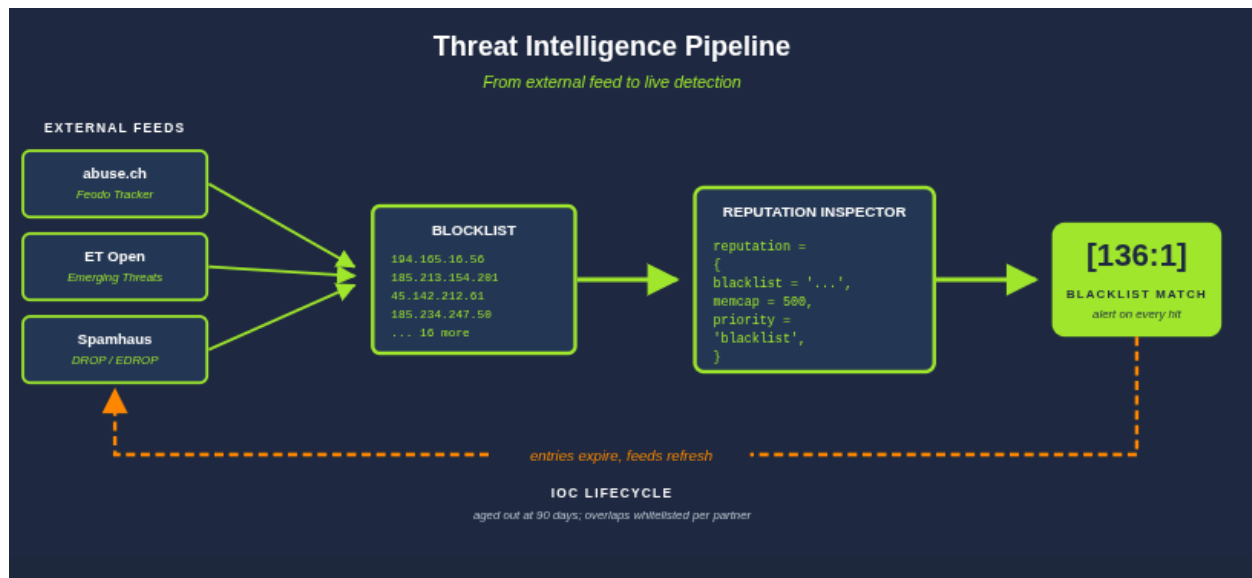
39887

After suppressing 10.14.22.199, how many alerts does Snort produce against task-04-noisy-traffic.pcap ?

52

```
ubuntu@tryhackme:~/lab/task-04$ sudo snort -c /etc/snort/snort.lua -R task-04-answer.rules -r task-04-noisy-traffic.pcap -A alert_fast -q | wc -l
39887
ubuntu@tryhackme:~/lab/task-04$ sudo snort -c /etc/snort/snort.lua -R task-04-answer.rules -r task-04-noisy-traffic.pcap -A alert_fast -q | wc -l
39887
ubuntu@tryhackme:~/lab/task-04$ nano task-04-suppress.lua
ubuntu@tryhackme:~/lab/task-04$ sudo snort -c /etc/snort/snort.lua -R task-04-answer.rules -r task-04-noisy-traffic.pcap -A alert_fast -q --tweaks task-04-suppress | wc -l
<REDACTED>
bash: ubuntu@tryhackme:~/lab/task-04$: No such file or directory
0
bash: syntax error near unexpected token `newline'
ubuntu@tryhackme:~/lab/task-04$ sudo snort -c /etc/snort/snort.lua -R task-04-answer.rules -r task-04-noisy-traffic.pcap -A alert_fast -q --tweaks task-04-suppress | wc -l
52
ubuntu@tryhackme:~/lab/task-04$
```

Task 5 - Threat Intelligence Integration



- `task-05-blocklist.txt`: a 20-entry IP blocklist aggregated from abuse.ch Feodo Tracker and Emerging Threats Open. Two of those entries are the Operation SILENT TRANSFER addresses we have been tracking, `194.165.16.56` (C2) and `185.213.154.201` (exfil).
- `task-05-ti-integration.pcap`: a 20 MB capture that contains ten outbound connections to addresses on the blocklist, mixed into legitimate HTTPS browsing.
- `task-05-snort.lua`: a staged config with the reputation inspector stanza already drafted but not yet enabled.
- `task-05-ti-integration.py`: the script used to generate the PCAP. You do not need to run it — it is included for transparency.

Let's look at the feed we are about to load. This is a plain text file, one IP per line, with comments that tell us where each entry came from and when it was added:

```

ubuntu@tryhackme:~/lab/task-05$ head -n 10 task-05-blocklist.txt
# Operation SILENT TRANSFER - internal TI, 2026-04-12
194.165.16.56      # Feodo Tracker - active C2
185.213.154.201   # ET Open - exfil infrastructure# External
feeds - abuse.ch Feodo Tracker, 2026-04-18
45.142.212.61

```

```
185.234.247.50
91.92.241.17
193.32.162.8
```

Enabling the Reputation Inspector

Open the staged config and find the reputation stanza:

```
ubuntu@tryhackme:~/lab/task-05$ sudo nano task-05-snort.lua
ubuntu@tryhackme:~/lab/task-05$
```

Scroll to the reputation block and edit it to match this:

```
reputation =
{
  blocklist = '/home/ubuntu/lab/task-05/task-05-blocklist.txt',
  memcap = 500,
  priority = 'blocklist',
  nested_ip = 'inner',
}
```

`blocklist` is the absolute path to our feed. `memcap` is the RAM budget for the loaded list, measured in megabytes. 500 MB is generous for 20 entries, but a production feed pulling from Spamhaus DROP and EDROP will easily exceed 100 MB after parsing. `Priority` determines which list wins when an IP address appears in both a blocklist and an allowlist. `nested_ip` decides which address to check when tunneled protocols carry an inner header. Save and exit.

Running the Capture: Run the config against the integration capture:

```
ubuntu@tryhackme:~/lab/task-05$ sudo snort -c task-05-snort.lua -r task-05-ti-integration.pcap -A alert_fast -q | wc -l
<REDACTED>
```

Looking at the output, each alert is a connection to an address on our feed. Let's see which SID they carry:

```
ubuntu@tryhackme:~/lab/task-05$ sudo snort -c task-05-snort.l
ua -r task-05-ti-integration.pcap -A alert_fast -q | head -n
1
04/22-09:14:02.118432 [**] [136:1:1] "(reputation) packet fro
m blacklisted source/destination IP" [**] [Priority: 2] {TCP}
10.14.22.88:51122 -> 194.165.16.56:443
```

The output shows `[136:1:1]`. That is the reputation inspector speaking. Every Snort inspector owns a `gen_id`, and `136` is the reputation. The `:1` is the `sig_id` for a blacklist match. When we tune around these alerts later, we reference `gen_id 136, sig_id 1`, not the `gen_id 1` that belongs to text rules.

IOC Lifecycle: Loading a feed is the easy part. Keeping it useful is the work that matters. Three things tend to break a blocklist over time if we leave it alone. Entries go stale. An IP that was C2 six months ago may now be reassigned to a legitimate SaaS customer on the same hosting provider. If we never age entries out, our blocklist slowly turns into a false positive machine. A simple fix is to timestamp each entry and drop anything older than a fixed window unless we re-validate it against a live feed. Feeds overlap with legitimate infrastructure. A Spamhaus DROP range occasionally catches a shared CDN edge node that also serves a partner's web traffic. When that happens, we add the partner's specific IP to the allowlist with `priority = 'whitelist'` in effect, or we exclude that range from the blocklist load. We do not delete the Spamhaus entry for everyone else.

Domains need different handling. The reputation inspector is IP-only. If our TI report names a domain such as `update.softpatch-cdn.com`, we pair the blocklist with a content rule that matches the domain inside DNS queries or TLS SNI. That way, the detection survives the C2 rotating IPs under the same name.

Answer the questions below

After enabling the reputation inspector and running it on `task-05-ti-integration.pcap`, how many alerts are produced?

20

```

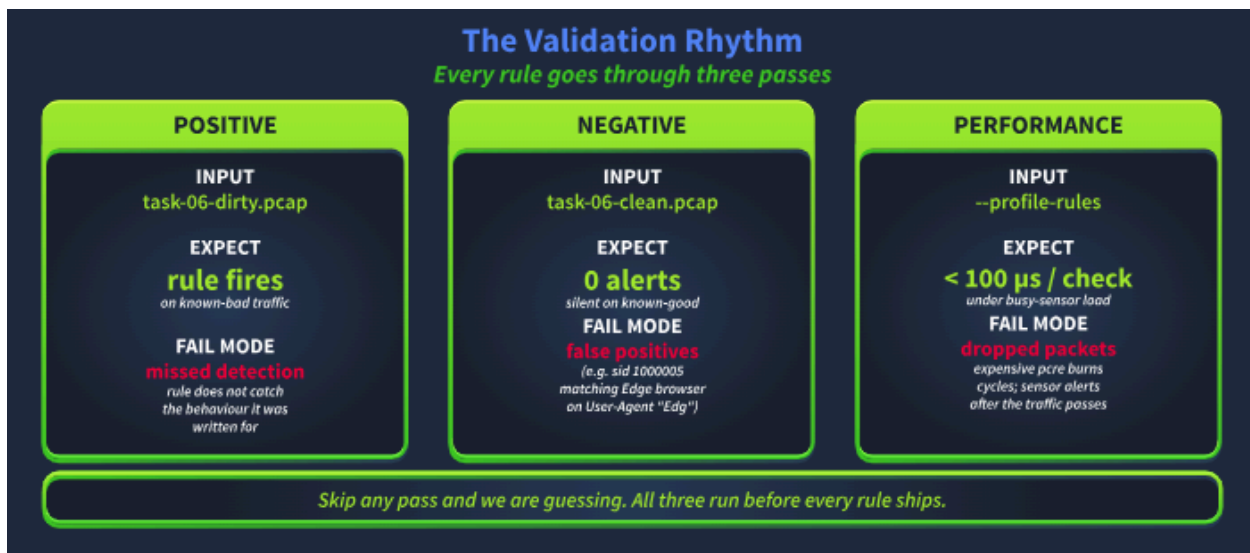
ess | wc -l
52
ubuntu@tryhackme:~/lab/task-04$ cd /home/ubuntu/lab/task-05
ubuntu@tryhackme:~/lab/task-05$ head -n 10 task-05-blocklist.txt
# Operation SILENT TRANSFER - internal TI, 2026-04-12
194.165.16.56      # Feodo Tracker - active C2
185.213.154.201   # ET Open - exfil infrastructure

# External feeds - abuse.ch Feodo Tracker, 2026-04-18
45.142.212.61
185.234.247.50
91.92.241.17
193.32.162.8

ubuntu@tryhackme:~/lab/task-05$ sudo nano task-05-snort.lua
ubuntu@tryhackme:~/lab/task-05$ sudo snort -c task-05-snort.lua -r task-05-ti-in
tegration.pcap -A alert_fast -q | wc -l
20
ubuntu@tryhackme:~/lab/task-05$

```

Task 6 - Validating Detection



- `task-06-ruleset.rules` : Five rules a junior analyst shipped last week. They all fire on the dirty PCAP. One of them also fires on clean traffic, which is the problem.
- `task-06-dirty.pcap` : aAcapture containing the full Operation SILENT TRANSFER intrusion slice. C2 beacon, DNS tunnel, lateral SMB, SNI hit.

- `task-06-clean.pcap`: Thirty minutes of ordinary Finance subnet traffic. HTTPS browsing, Windows Update, Teams, and Microsoft Edge on a handful of workstations.
- `task-06-answer.rules`: The fixed version we will compare against at the end.

The Three-Pass Validation Rhythm: Every rule we ship goes through the same three passes. The positive test proves the rule fires on the traffic we wrote it for. The negative test shows that the rule remains silent on traffic we do not care about. The performance test proves the rule is not dragging the sensor down under load. Skip any one of these, and we are guessing.

Positive Test: Dirty PCAP

```
ubuntu@tryhackme:~/lab/task-06$ sudo snort -c /etc/snort/snort.lua -R task-06-ruleset.rules -r task-06-dirty.pcap -A alert_fast -q | awk '{print $3}' | sort | uniq -c
      3 [1:1000001:1]
     10 [1:1000002:1]
       5 [1:1000003:1]
       4 [1:1000004:1]
       3 [1:1000005:1]
```

Looking at the output, all five SIDs in the ruleset fire at least once: `1000001` (HTTP GET), `1000002` (C2 beacon), `1000003` (DNS tunnel), `1000004` (SMB lateral), `1000005` (Edg user-agent hunt). Positive test passes. The rules do catch the traffic they were written for.

Negative Test: Clean PCAP:

```
ubuntu@tryhackme:~/lab/task-06$ sudo snort -c /etc/snort/snort.lua -R task-06-ruleset.rules -r task-06-clean.pcap -A alert_fast -q | wc -l
<REDACTED>
```

Looking at the output, we should have had zero. Every hit here is a false positive that the SOC will have to triage. Let's find out which rule is responsible: Group Alerts By SID

```
ubuntu@tryhackme:~/lab/task-06$ sudo snort -c /etc/snort/snort.lua -R task-06-ruleset.rules -r task-06-clean.pcap -A alert_fast -q | grep -oE '\[1:[0-9]+:[0-9]+\]' | sort | uniq -c
?? [1:1000005:1]
```

The output shows that all hits belong to `[1:1000005:1]`. One rule is to produce the entire false-positive stream. Open the `task-06-ruleset.rules` ruleset and look at what sid `1000005` was written to match:

```
alert tcp $HOME_NET any -> $EXTERNAL_NET any (
  msg:"SILENT TRANSFER - Edg User-Agent hunt";
  flow:to_server,established;
  content:"User-Agent|3a 20|", nocase;
  content:"Edg", within 30;
  sid:1000005; rev:1;)
```

The author saw `"Edg"` in a threat report and wrote a rule for it. The problem is that Microsoft Edge sends `User-Agent: ... Edg/123.0.0.0` on every request. The rule does not distinguish between the malware's three-letter indicator and the legitimate browser string. Every Edge user on the Finance subnet trips the rule.

Fixing the False Positive: The fix is to anchor the match on something unique to the malware. The original report said the indicator was the literal token `EdgBot` as part of a custom user-agent, not the `Edg/` version tag. Edit the rule:

```
content:"EdgBot", within 30;
```

Save and re-run the negative test: Negative Test After Fix

```
ubuntu@tryhackme:~/lab/task-06$ sudo snort -c /etc/snort/snort.lua -R task-06-answer.rules -r task-06-clean.pcap -A alert_fast -q | wc -l
0
```

The output shows 0 alerts. Re-run the positive test to confirm the fix did not break detection on the dirty capture: Re-Verify Positive Test

```
ubuntu@tryhackme:~/lab/task-06$ sudo snort -c /etc/snort/snort.lua -R task-06-answer.rules -r task-06-dirty.pcap -A alert_fast -q | grep -c 1000005
<REDACTED>
```

Looking at the output, the C2 traffic still triggers the rule while the Edge browser traffic is completely silent. The rule went from unusable to production-ready with one content change.

Performance Test: Before we ship, measure the cost. Snort will profile every rule and tell us which ones are expensive:

```
ubuntu@tryhackme:~/lab/task-06$ sudo snort -c /etc/snort/snort.lua -R task-06-answer.rules -r task-06-dirty.pcap --lua "profiler = { rules = { count = 10, sort = 'total_time' } }" -q
rule profile (worst 10, sorted by total_time)#      gid      sid rev      checks matches alerts time (us) avg/check avg/match avg/non-match
=====
=====
1          1 1000002 1          20          10          10          73
3          6          0
2          1 1000005 2           9           3           3          28
3          4
```

The output shows checks, matches, and microseconds per check for each SID. A `pcrc`-heavy rule that burns thousands of microseconds per check against a busy sensor will drop packets before it alerts. Could you flag anything at the top of the list for a rewrite, and keep the profile output with the rule's version history so future changes have a baseline?

Answer the questions below

How many false positive alerts fire when running the shipped ruleset against `task-06-clean.pcap`?

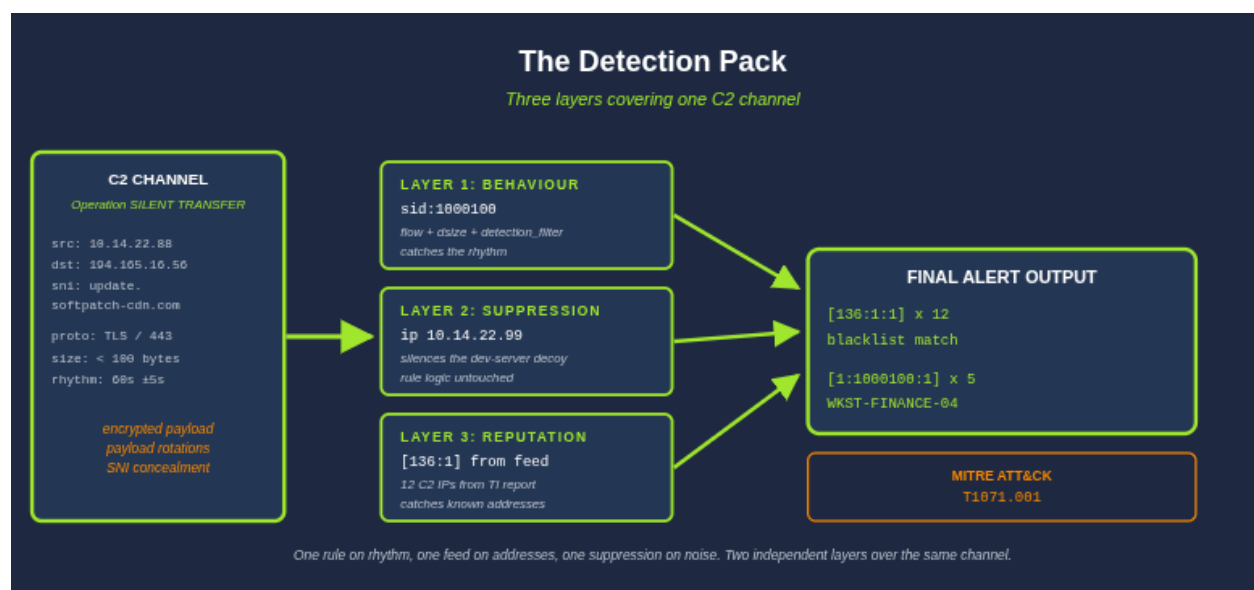
15

After fixing sid 1000005 and re-running against `task-06-dirty.pcap`, how many times does the corrected rule fire?

3

```
ubuntu@tryhackme:~/lab/task-06$ sudo snort -c /etc/snort/snort.lua -R task-06-ruleset.rules -r task-06-dirty.pcap -A alert_fast -q | awk '{print $3}' | sort | uniq -c
      3 [1:1000001:1]
     10 [1:1000002:1]
      5 [1:1000003:1]
      4 [1:1000004:1]
      3 [1:1000005:1]
ubuntu@tryhackme:~/lab/task-06$ sudo snort -c /etc/snort/snort.lua -R task-06-ruleset.rules -r task-06-clean.pcap -A alert_fast -q | wc -l
15
ubuntu@tryhackme:~/lab/task-06$ sudo snort -c /etc/snort/snort.lua -R task-06-ruleset.rules -r task-06-clean.pcap -A alert_fast -q | grep -oE '\[1:[0-9]+:[0-9]+\]' | sort | uniq -c
     15 [1:1000005:1]
ubuntu@tryhackme:~/lab/task-06$ sudo snort -c /etc/snort/snort.lua -R task-06-answer.rules -r task-06-clean.pcap -A alert_fast -q | wc -l
0
ubuntu@tryhackme:~/lab/task-06$ sudo snort -c /etc/snort/snort.lua -R task-06-answer.rules -r task-06-dirty.pcap -A alert_fast -q | grep -c 1000005
3
ubuntu@tryhackme:~/lab/task-06$
```

Task 7 - Detection Challenge



- `task-07-challenge.pcap` : a 310 MB perimeter capture covering a 30-minute window across the Finance subnet.
- `task-07-ti-report.md` : a one-page threat intelligence report on the suspected malware family.
- `task-07-challenge-snort.lua` : a clean Snort 3 config with the reputation inspector stanza primed but no blocklist loaded.
- `task-07-challenge-local.rules` : an empty ruleset. We fill it ourselves.
- `task-07-challenge-blocklist.txt` : an empty file we will populate with C2 addresses from the TI report.
- `task-07-challenge-threshold.conf.lua` : an empty Lua suppression file we will edit during tuning.

Reading the TI Report: Before we touch the PCAP, could you read what intelligence already has on the malware family? Two minutes here saves an hour of blind hunting:

```
ubuntu@tryhackme:~/lab/task-07$ cat task-07-ti-report.md
# Internal Threat Intelligence Brief

**Subject:** Operation SILENT TRANSFER - Finance segment targeting
**Issued:** 2026-04-22 08:10 UTC
**Classification:** TLP:AMBER
**Analyst:** TI-02 (EMEA desk)
**Ticket:** NFG-2145
```

The report names the family as `SILENT TRANSFER`, maps the beaconing behavior to MITRE ATT&CK `T1071.001` (Application Layer Protocol: Web Protocols), lists twelve C2 addresses, including `194.165.16.56` and `185.213.154.201`, and describes the beacon pattern: a TLS connection every 60 seconds plus or minus 5, with payloads under 100 bytes, SNI set to `update.softpatch-cdn.com`. That is enough to write the rule and to populate the blocklist.

Triage the Capture: Top Internal Talkers

```
ubuntu@tryhackme:~/lab/task-07$ sudo tcpdump -r task-07-challenge.pcap -nn 'src net 10.14.22.0/24' 2>/dev/null | awk '{print $3}' | cut -d. -f1-4 | sort | uniq -c | sort -rn | head
300 10.14.22.77
200 10.14.22.99
40 10.14.22.88
4 10.14.22.45
4 10.14.22.156
```

Looking at the output, three internal addresses rise above the noise: `10.14.22.88` (WKST-FINANCE-04), `10.14.22.77` (a misconfigured resolver chewing through DNS queries), and `10.14.22.99` (a dev server pinging its own health endpoint every 10 seconds). Two of those three are decoys; the third is our target. Triage narrows the field. Running the detection is what confirms which host is actually malicious.

Writing the Beacon Rule: From the TI report, we know the beacon shape is small, regular, and outbound TLS. Open the empty ruleset `task-07-challenge-local.rules` and drop in the detection:

```
alert tcp $HOME_NET any -> $EXTERNAL_NET 443 (
  msg:"SILENT TRANSFER - C2 beacon rhythm";
  flow:to_server,established;
  dsize:<100;
  detection_filter:track by_src, count 5, seconds 360;
  metadata:mitre_attack T1071.001, author "saqib", ticket "NFG-2145";
  sid:1000100; rev:1;)
```

Save the rule and run it against the challenge capture: Run the Beacon Rule

```
ubuntu@tryhackme:~/lab/task-07$ sudo snort -c task-07-challenge-snort.lua -R task-07-challenge-local.rules -r task-07-challenge.pcap -A alert_fast -q | grep -oE '10\.14\.22\.[0-9]+' | sort | uniq -c | sort -rn
<REDACTED> <REDACTED>
<REDACTED> 10.14.22.88
```

The output shows two source addresses firing the rule. `10.14.22.88` is WKST-FINANCE-04, our actual target, beaconing every 60 seconds. `<REDACTED>` is the dev server health check: its alerts come from a polling loop, not a beacon. The other Finance hosts visible in the tcpdump output, `10.14.22.45` and `10.14.22.156`, each have only 4 qualifying packets in the capture window, which falls below the `detection_filter threshold of 5`, so the rule never fires on them. The signal is `10.14.22.88`; the noise is `<REDACTED>`. Suppression is how we separate them.

Suppressing the Decoy: Edit `task-07-challenge-threshold.conf.lua` and add the suppression entry for the health-check host:

```
suppress =
{
  { gid = 1, sid = 1000100, track = 'by_src', ip = '10.14.22.99' }
}
```

Save and re-run with the suppression file loaded: Re-run After Suppression

```
ubuntu@tryhackme:~/lab/task-07$ sudo snort -c task-07-challenge-snort.lua -R task-07-challenge-local.rules -r task-07-challenge.pcap -A alert_fast -q --tweaks task-07-challenge-threshold.conf.lua | grep 1000100 | wc -l
<REDACTED>
```

The output shows only alerts from `10.14.22.88`. With `10.14.22.99` silenced, WKST-FINANCE-04 stands alone as the only host sustaining the beacon rhythm across the capture window. Suppression removed the dominant noise source without touching the rule's detection logic, exactly the pattern we established in Task 4.

Loading the Blocklist: The TI report lists twelve C2 addresses in its infrastructure table. Rather than copying them manually, we can extract every external IP address from the report in a single pipeline and write them directly into the blocklist file: Extract C2 IPs from TI Report

```
ubuntu@tryhackme:~/lab/task-07$ grep -oE '^\\|[[[:space:]]*[0-9]{1,3}\\.[0-9]{1,3}\\.[0-9]{1,3}\\.[0-9]{1,3}' task-07-ti-report.md \
```

```
| grep -oE '[0-9]{1,3}\.[0-9]{1,3}\.[0-9]{1,3}\.[0-9]{1,3}'
\
| grep -vE '^(10\.|172\.([6-9]|2[0-9]|3[01])\.|192\.168
\.)' \
> task-07-challenge-blocklist.txt
```

The pipeline has three stages. The first `grep` targets only lines that open a markdown table cell (`|` followed by an IP), scoping the extraction to the infrastructure table rather than prose paragraphs. The second strips the pipe and whitespace, leaving a bare IP per line. The third filters out RFC 1918 private ranges — the report also tables the affected internal hosts, which we do not want in a C2 blocklist. Verify the count matches what the report advertises: Verify Blocklist

```
ubuntu@tryhackme:~/lab/task-07$ wc -l task-07-challenge-block
list.txt
12 task-07-challenge-blocklist.txt
```

Twelve lines, one IP per line. Now open `task-07-challenge-snort.lua` and point the reputation inspector stanza at the blocklist file:

```
reputation =
{
  blocklist = '/home/ubuntu/lab/task-07/task-07-challenge-blocklist.txt',
  memcap = 500,
  priority = 'blocklist',
  nested_ip = 'inner',
}
```

Re-run and look at the full alert summary, grouped by signature: Final Alert Summary

```
ubuntu@tryhackme:~/lab/task-07$ sudo snort -c task-07-challen
ge-snort.lua -R task-07-challenge-local.rules -r task-07-chal
lenge.pcap -A alert_fast -q --tweaks task-07-challenge-thresh
old.conf.lua | grep -oE '\[[0-9]+:[0-9]+:[0-9]+\]' | sort | u
```

```
niq -c  
<REDACTED> [136:1:1]
```

The output shows a single alert grouping: the reputation inspector. When the reputation inspector encounters a flow to a blocklisted C2 address, it acts on that packet before the detection engine evaluates the beacon rule, which is why `[1:1000100:1]` does not appear in the final summary. The reputation alerts confirm the blocklist is catching every connection attempt to the C2 infrastructure, including `194.165.16.56` and `185.213.154.201`. The two layers operate on the same channel but serve different purposes: the reputation feed handles known-bad infrastructure, and the behavior rule provides forward coverage for any C2 that rotates to an address not yet in the feed.

What Shipped: We shipped four deliverables: a behavior rule, a suppression entry, a reputation feed, and populated local `rules`. That is the detection pack the brief called for. The same pack used for next week's capture will still catch the same beacon, even if the attacker rotates to a new C2 address, because the rule does not care about the address. It cares about the rhythm.

Answer the questions below

Before adding the suppression entry, how many alerts does the beacon rule produce?

170

Which source IP on the Finance subnet is responsible for the beacon-rule noise that must be suppressed?

10.14.22.99

After adding the suppression entry, how many SID 1000100 alerts remain?

25

After loading the blocklist, how many reputation inspector alerts appear in the final summary?

12

Task 8 - Conclusion

In the first room, an attacker exfiltrated 847 MB over QUIC, and the SOC only caught it after the fact. Underneath that exfiltration sat a 48-hour TLS C2 beacon for which no rule had been written. Both gaps are now covered.

The `local.rules` file we shipped detects the beacon on rhythm rather than a fixed C2 address, so it remains useful even after the attacker rotates infrastructure. The suppression and threshold entries we added keep the alert volume readable, so the SOC can actually see what fires. The reputation feed catches the known-bad IPs we already have intelligence on. Every rule went through positive, negative, and performance testing before it left staging.

The Detection Engineering Mindset: Three habits separate operating Snort from engineering detection with it.

- **Detect behavior, not signatures.** Attackers swap tools more often than they swap tradecraft. A rule built around the attacker's behavior continues to work even when the domain rotates, the payload is recompiled, or the channel switches to QUIC. A rule built around a specific byte string fails the moment the operator edits a line of code.
- **Tune before you ship.** A noisy rule that nobody reads is worse than no rule at all. Every false positive crowds out an alert that actually matters. A well-running SOC does not just write rules; it tunes them, thresholds them, and retires them when they stop working.
- **Validate every change.** The three-pass cycle (positive test, negative test, performance profile) applies to every rule before it leaves staging. Running the cycle takes minutes. Skipping it is how outages happen.

What Snort Still Cannot See: Snort fires on rule matches. Anything our rules do not describe stays invisible to it. Snort cannot see connections that no rule matches against, encrypted channels where there is no plaintext payload to inspect, or first-appearance traffic that no threat feed has flagged yet. All three sit outside this room's coverage.

When we need to investigate traffic that did not produce an alert, Snort alone is not enough. That is where Zeek comes in. Zeek writes a structured log line for every connection, every DNS query, and every TLS handshake, regardless of

whether a rule fired. When we need to know what crossed the wire without alerting, the Zeek logs are the starting point.